JUnit5: Parameterized Tests

Definition

It is used to run the **same test** (but **with different inputs**) multiple times.

Single Parameter Parameterized Tests

provides a single parameter to the test, running it multiple time with different values.

@ValueSource

@EnumSource

@ValueSource

```
public class MyStringUtilTest {

    @ParameterizedTest
    @ValueSource(strings = {"civic", "level", "madam"})
    void testPalindrome(String word) {
        assertTrue(MyStringUtil.isPalindrome(word));
    }
}
```

Test will run 3 times, each with different parameter.
1. civic
2. Level
3. madam

Output:

```
Run    MyStringUtilTest.testPalindrome
MyStringUtilTest (concepts.ParameterisedTest)   12 ms
  testPalindrome(String)   12 ms
    [1] word=civic   10 ms
    [2] word=level
    [3] word=madam   2 ms
Tests passed: 3 of 3 tests – 12 ms
```

Supported Types	Example
strings	@ValueSource(strings={"civic","level","madam"})
ints	@ValueSource(ints = {1, 2, 3})
shorts	@ValueSource(shorts = {1, 2, 3})
bytes	@ValueSource(bytes = {1, 2, 3})

Concept && Coding

longs	@ValueSource(longs = {1L, 2L, 3L})
floats	@ValueSource(floats = {1.1f, 2.2f}
doubles	@ValueSource(doubles = {1.1, 2.2})
classes	@ValueSource(classes = {String.class, Integer.class})

Implicit Conversions:

JUnit 5 automatically **converts between compatible types**.

```
//implicit conversion
@ParameterizedTest
@ValueSource(strings = {"1", "2", "3"})
void testImplicitIntConversion(int num) {
    assertTrue( condition: num >= 1);
}
```

String is converted to int implicitly.

If it can not, it throws **ParameterResolutionException**

```
@ParameterizedTest
@ValueSource(strings = {"1", "2", "hello"})
void testImplicitIntConversion(int num) {
    assertTrue( condition: num >= 1);
}
```

```
org.junit.jupiter.api.extension.ParameterResolutionException: Error converting parameter at index 0: Failed to convert String "hello" to type int

    at java.base/java.util.Optional.ifPresent(Optional.java:178) <31 internal lines>
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
Caused by: org.junit.jupiter.params.converter.ArgumentConversionException: Failed to convert String "hello" to type int
    ... 34 more
```

Explicit Conversions:

@ConvertWith annotation along with **ArgumentConverter** Interface is used for manually defining the conversion logic.

```
//explicit conversion
@ParameterizedTest
@ValueSource(strings = {"hello", "world"})
void testExplicitConversion(@ConvertWith(StringUpperCaseConverter.class) String word) {
    // All words are converted to uppercase before assertion
    assertEquals(word, word.toUpperCase());
}
```

- *@ValueSource provides raw string values "hello", "world".*
- *@ConvertWith calls our converter **before** the test parameter is injected.*
- *Inside the converter, we define any custom logic .*
- *The test sees only the converted value.*

Concept && Coding

```java
public class StringUpperCaseConverter implements ArgumentConverter {

    @Override
    public Object convert(Object source, ParameterContext parameterContext) throws ArgumentConversionException {

        if (source instanceof String && parameterContext.getParameter().getType() == String.class) {
            return ((String) source).toUpperCase();
        }
        throw new IllegalArgumentException("Conversion failed");
    }
}
```

ParameterContext object has all the info about the particular parameter:
- *Its position in parameter list*
- *Its type*
- *Which method or constructor declared this variable*

using @ValueSource, we can not provide null value, so there is specialized annotations for it:

- **@NullSource**: feed NULL value
- **@EmptySource**: feed empty value to object
- **@NullAndEmptySource**: combined both

```java
@ParameterizedTest
@NullAndEmptySource
@ValueSource(strings = {"civic", "level", "madam"})
void testPalindrome(String word) {
    assertTrue(MyStringUtil.isPalindrome(word));
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)   12 ms
  testPalindrome(String)                        12 ms
    [1] word=null                               11 ms
    [2] word=                                    1 ms
    [3] word=civic
    [4] word=level
    [5] word=madam

Tests failed: 1, passed: 4 of 5 tests – 12 ms
```

@EnumSource:

```java
//Enum Parameter : All Enum values
@ParameterizedTest
@EnumSource(value = Day.class)
void testEnumValues(Day day) {
    assertTrue( condition: day.name().length() > 0);
}
```

Concept && Coding

```java
public enum Day {
    SUNDAY,
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)  13 ms
  testEnumValues(Day)  13 ms
    [1] day=SUNDAY  13 ms
    [2] day=MONDAY
    [3] day=TUESDAY
    [4] day=WEDNESDAY
    [5] day=THURSDAY
    [6] day=FRIDAY
    [7] day=SATURDAY

Tests passed: 7 of 7 tests – 13 ms
/Users/shrayanshjain/Library/Java/JavaVirtua

Process finished with exit code 0
```

Only Specific Enum Values:

```java
//Enum Parameter : specific Enum values
@ParameterizedTest
@EnumSource(value = Day.class, names = {"MONDAY", "TUESDAY"}, mode = EnumSource.Mode.INCLUDE)
void testEnumValues(Day day) {
    assertTrue( condition: day.name().length() > 0);
}
```

default

```
MyStringUtilTest (concepts.ParameterisedTest)  11 ms
  testEnumValues(Day)  11 ms
    [1] day=MONDAY  11 ms
    [2] day=TUESDAY

Tests passed: 2 of 2 tests – 11 ms
/Users/shrayanshjain/Library/Java/Java

Process finished with exit code 0
```

Using Exclude mode:

```java
//Enum Parameter : specific Enum values
@ParameterizedTest
@EnumSource(value = Day.class, names = {"MONDAY", "TUESDAY"}, mode = EnumSource.Mode.EXCLUDE)
void testEnumValues(Day day) {
    assertTrue( condition: day.name().length() > 0);
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)  12 ms
  testEnumValues(Day)  12 ms
    [1] day=SUNDAY  12 ms
    [2] day=WEDNESDAY
    [3] day=THURSDAY
    [4] day=FRIDAY
    [5] day=SATURDAY

Tests passed: 5 of 5 tests – 12 ms
/Users/shrayanshjain/Library/Java/Ja

Process finished with exit code 0
```

Concept && Coding

Multiple Parameter Parameterized Tests

provides multiple parameter to the test, running it multiple time with different values.

@CsvSource

@CsvFileSource

@CsvSource:

Using this, we can provide multiple parameters inline.

```
@ParameterizedTest
@CsvSource(value = {"1,2,3", "5,5,10"}, delimiter = ',')
void testCSVSource(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

',' is default, so even if we skip this. It will work fine.
@CsvSource(value={"1,2,3","5,5,10"})

First line used as csv header by setting the *useHeadersInDisplayName* attribute to true

```
@ParameterizedTest
@CsvSource(
        delimiter = '|',
        useHeadersInDisplayName = true,
        value = {
                "Val1|Val2|ExpectedVal",
                "1|2|3",
                "5|5|10",
                "4|1|5"
        }
)
void testCSVSourceWithHeader(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

- By default, "," is used as delimiter.
- So what if our Value itself need "," then we can enclosed our value withing ' (single quotes)

```
@ParameterizedTest
@CsvSource(value = {"hello,5", "world,5", "'Delhi,India',11"})
void testCSVSourceWithSingleQuotes(String word, int length) {
    assertEquals(length, word.length());
}
```

Example Input	Result
@CsvSource({ "hello, world" })	1st argument: hello 2nd argument: world

Concept && Coding

@CsvSource({ "hello, 'Delhi, India'" })	1st argument: hello 2nd argument: Delhi,India
@CsvSource({ "hello, '" }) Single quote start and end	1st argument: hello 2nd argument: '' (empty)
@CsvSource({ "hello, " })	1st argument: hello 2nd argument: null
@CsvSource(value = { "hello, world, N/A" }, nullValues = "N/A")	1st argument: hello 2nd argument: world 3rd argument: null
@CsvSource(value = { " hello, world" }, ignoreLeadingAndTrailingWhitespace = false) by default ignoreLeadingAndTrailingWhitespace is true.	1st argument: ' hello' 2nd argument: ' world' note there is a space before hello and world, its not trimmed.

@CsvFileSource:

When input data is large, then we can use @csvFileSource to load test data from an external csv file instead of writing it inline.

```
@ParameterizedTest
//file is read from path: /src/test/resources
@CsvFileSource(resources = "/inputData.csv", numLinesToSkip = 1)
void testCSVFileSource(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

src/test/resources/inputData.csv

```
val1,val2,expected
1,2,3
5,5,10
```

Output:

```
MyStringUtilTest (concepts.ParameterisedTest)   14 ms
  testCSVFileSource(int, int, int)              14 ms
    [1] a=1, b=2, expected=3                    14 ms
    [2] a=5, b=5, expected=10

Tests passed: 2 of 2 tests – 14 ms
/Users/shrayanshjain/Library/Java/JavaVirtualM

Process finished with exit code 0
```

Instead of "numLinesToSkip", we can also use "*useHeadersInDisplayName*" attribute to true

Concept && Coding

```
@ParameterizedTest
//file is read from path: /src/test/resources
@CsvFileSource(resources = "/inputData.csv", useHeadersInDisplayName = true)
void testCSVFileSource(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)                    13 ms
  testCSVFileSource(int, int, int)                               13 ms
    [1] a=val1 = 1, b=val2 = 2, expected=expected = 3            13 ms
    [2] a=val1 = 5, b=val2 = 5, expected=expected = 10
```

But don't use both, as it will skip 2 lines:

- 1 line skipped by = numLinesToSkip
- Another by *useHeadersInDisplayName, it will treat the line as header and skip it*

```
@ParameterizedTest
//file is read from path: /src/test/resources
@CsvFileSource(resources = "/inputData.csv", numLinesToSkip = 1, useHeadersInDisplayName = true)
void testCSVFileSource(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

Input File:

```
1  val1,val2,expected
2  1,2,3
3  5,5,10
```

Output (only 1 test data runs, and first 2 rows got skipped):

```
MyStringUtilTest (concepts.ParameterisedTest)                    13 ms
  testCSVFileSource(int, int, int)                               13 ms
    [1] a=val1 = 5, b=val2 = 5, expected=expected = 10           13 ms
```

```
Tests passed: 1 of 1 test – 13 ms
/Users/shrayanshjain/Library/Java/J
Process finished with exit code 0
```

Object and Complex parameters

@MethodSource:

- Provides test data from the static method in the same or different class.
- This static method, can return any of these types:

  - Stream<T> - for single parameter
  - Stream<Arguments> - for multiple parameter
  - Collection<T> - for single parameter
  - T[] - for single parameter
  - Object[] - for single parameter
  - Object[][] - for multiple parameter

    Etc..

Example:

Concept && Coding

- Stream<T> :

We can pass anything like String, Integer, Object etc.

```
@ParameterizedTest
@MethodSource("testDataForStreamSingleParameter")
void testMethodSourceStreamSingleParameter(String word) {
    assertTrue(MyStringUtil.isPalindrome(word));
}

static Stream<String> testDataForStreamSingleParameter(){
    return Stream.of( ...values: "civic", "radar");
}
```

- Collection<T>

When the test data method name is missed, then by-default it looks for static method with name same as test method name.

```
@ParameterizedTest
@MethodSource()
void testMethodSourceCollectionSingleParameter(String word) {
    assertTrue(MyStringUtil.isPalindrome(word));
}

static List<String> testMethodSourceCollectionSingleParameter(){
    return List.of("civic", "radar", "madam");
}
```

- T[]

```
@ParameterizedTest
@MethodSource("testDataForArraySingleParameter")
void testMethodSourceArraySingleParameter(String word) {
    assertTrue(MyStringUtil.isPalindrome(word));
}

static String[] testDataForArraySingleParameter(){
    return new String[] {"civic", "madam"};
}
```

- Object[]

Object parameters too we can feed.

```
@ParameterizedTest
@MethodSource("userTestData")
void testMethodSourceObjectArraySingleParameter(User user) {
    assertTrue( condition: user.age > 18);    }

static Object[] userTestData(){
    return new Object[] {
            new User( name: "A",  age: 20),
            new User( name: "B",  age: 21)
    };
}
```

- Stream<Arguments> :

```
@ParameterizedTest
@MethodSource("testDataForStreamMultipleParameter")
void testMethodSourceStreamMultipleParameter(String word, int expectedLength) {
    assertEquals(expectedLength, word.length());
}

static Stream<Arguments> testDataForStreamMultipleParameter(){
    return Stream.of(
            Arguments.of( ...arguments: "hello", 5),
            Arguments.of( ...arguments: "world", 5));
}
```

Also, when Test data static method is present in different class, then we need to provide fully qualified path

```
@ParameterizedTest
@MethodSource("concepts.ParameterisedTest.User#testDataForStreamMultipleParameter2")
void testMethodSourceStreamMultipleParameter2(String word, int expectedLength) {
    assertEquals(expectedLength, word.length());
}
```

Concept && Coding

@ArgumentsSource:

Somewhat similar to @MethodSource, but here instead of static method, we create a separate class itself.

Which provides more flexibility, when we have complex test input data.

Argument - represent one set of parameter of our test method.

Example:

Arguments.of(1, 2, 3) //multiple parameter

Arguments.of("Hello") // Single parameter

Arguments.of(userObj) // Object parameter

```
@ParameterizedTest
@ArgumentsSource(UserArgumentProvider.class)
void testCustomSource(User user) {
    assertNotNull(user.name);
}
```

```
Has info about :
- Test method, test class, instance, annotations, parameters etc..

public class UserArgumentProvider implements ArgumentsProvider {

    @Override
    public Stream<? extends Arguments> provideArguments(ExtensionContext context) {
        return Stream.of(
                Arguments.of(new User( name: "A",  age: 25)),
                Arguments.of(new User( name: "B",  age: 30))
        );
    }
}
```

@ArgumentsAccessor:

There could be a case, when we have so many multiple parameters in our method signature like:

Void testWithMultipleParameter(String arg1, String arg2, String arg3, int arg4, int arg5, int arg6.....){

}

One way to resolve this is: **@ArgumentsAccessor**

Concept && Coding

```java
@ParameterizedTest
@CsvSource({
        "A, 25",
        "B, 30"
})
void testWithArgumentsAccessor(ArgumentsAccessor accessor) {
    String name = accessor.getString( index: 0);
    int age = accessor.getInteger( index: 1);
    User userObj = new User(name, age);

    assertTrue( condition: userObj.age>18);
}
```

@ArgumentsAggregator:

Automatically maps multiple parameters into an object.

```java
@ParameterizedTest
@CsvSource({
        "A, 25",
        "B, 30"
})
void testAggregatorUser(@AggregateWith(UserAggregator.class) User user) {
    assertTrue( condition: user.age > 20);
}
```

```java
class UserAggregator implements ArgumentsAggregator {

    @Override
    public Object aggregateArguments(ArgumentsAccessor accessor, ParameterContext context) {
        return new User(accessor.getString( index: 0), accessor.getInteger( index: 1));
    }
}
```

This is similar to @ConvertWith (which we have seen at start), but that was only for single parameter, and @AggregateWith is for multiple parameter.

Customizing Display Names:

We can control the display name for each invocation:

Placeholder	meaning
{index}	Current invocation number
{arguments}	Comma separated list of all arguments
{0}, {1}, {2}....	Individual argument placeholders

Concept && Coding

Single Parameter:

```java
@ParameterizedTest(name = "Test {index}: Testing palindrome for {0}")
@ValueSource(strings = {"civic", "madam"})
void testDisplayNameSingleParameter(String word) {
    assertTrue(MyStringUtil.isPalindrome(word));
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)                12 ms
  testDisplayNameSingleParameter(String)                     12 ms
    Test 1: Testing palindrome for civic                     12 ms
    Test 2: Testing palindrome for madam

Tests passed: 2 of 2 tests – 12 ms
/Users/shrayanshjain/Library/Java/
Process finished with exit code 0
```

Multiple Parameter:

```java
@ParameterizedTest(name = "Test {index} : {arguments}")
@CsvSource(value = {"1,2,3", "5,5,10"}, delimiter = ',')
void testDisplayNameMultipleParameter(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)                13 ms
  testDisplayNameMultipleParameter(int, int, int)            13 ms
    Test 1 : 1, 2, 3                                         13 ms
    Test 2 : 5, 5, 10
```

```java
@ParameterizedTest(name = "Test {index} : {0} + {1} = {2}")
@CsvSource(value = {"1,2,3", "5,5,10"}, delimiter = ',')
void testDisplayNameMultipleParameterIndividually(int a, int b, int expected) {
    assertEquals(expected, actual: a + b);
}
```

```
MyStringUtilTest (concepts.ParameterisedTest)                14 ms
  testDisplayNameMultipleParameterIndividually(int, int, int)  14 ms
    Test 1 : 1 + 2 = 3                                       14 ms
    Test 2 : 5 + 5 = 10
```

Code link:

Junit5Lectures/src/test/java/concepts/ParameterisedTest · main · shrayansh jain / JUnit5AndMockito · GitLab

Concept && Coding